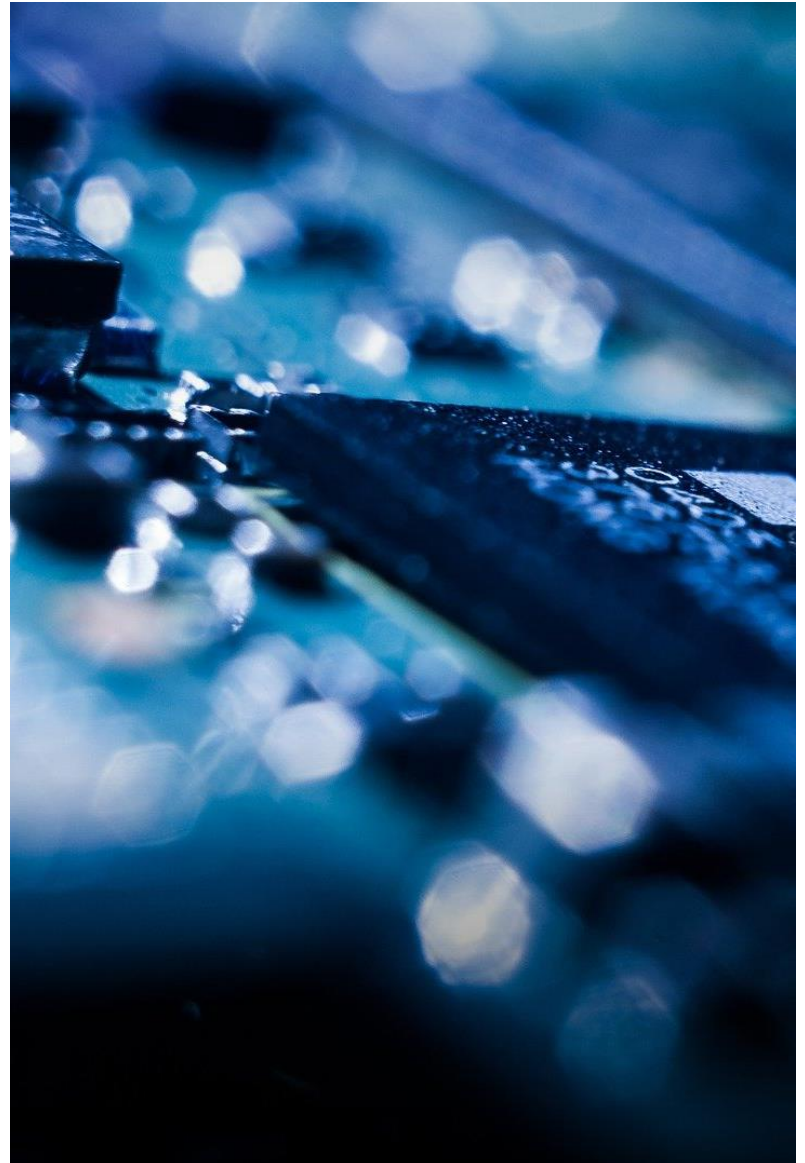


Programmierung eingebetteter Systeme

Zahlensysteme und Rechnen mit
Binärzahlen

Prof. Dr. Henning Trsek

Technische Hochschule Ostwestfalen-Lippe
Fachbereich Elektrotechnik und Technische Informatik
inIT - Institut für industrielle Informationstechnik
henning.trsek@th-owl.de



Überblick – Zahlensysteme

- ▶ Assembler-Direktiven
- ▶ Additionsbefehle
- ▶ Zahlendarstellung
- ▶ Vergleichen und Auswertung der Flags
- ▶ Flussdiagramme
- ▶ Schleifenkontrolle
- ▶ Addition von Zahlen > 32 Bit

Assembler-Direktiven

Assembler-Direktiven

- ▶ Anweisungen an den GNU-Assembler
- ▶ Direktiven beginnen mit einem Punkt, gefolgt von einem Schlüsselwort

Assembler-Direktiven

- ▶ Anweisungen an den GNU-Assembler
- ▶ Direktiven beginnen mit einem Punkt, gefolgt von einem Schlüsselwort

Beispiel:

```
.syntax unified  
.global asm2a  
.text
```

```
asm2a:    adr r0, tab1
```

...

```
table:    .word 0x11111111, 0x22222222, 0x33333333  
          .word 0x44444444, 0x55555555, 0x66666666
```

Assembler-Direktiven

```
<label> .byte <data>
```

```
<label> .word <data>
```

Beispiel:

...

```
tab1: .byte 0x11, 0x22, 0x33, 0x44
```

```
      .byte 0x55, 0x66, 0x77, 0x88
```

...

```
tab2: .word 0x11111111, 0x22222222
```

```
      .word 0x44444444, 0x55555555
```

```
      .word 0x55555555, 0x66666666
```

Assembler-Direktiven

Beispiel:

...
tab1: .byte 0x11, 0x22, 0x33, 0x44
 .byte 0x55, 0x66, 0x77, 0x88

...
tab2: .word 0x11111111, 0x22222222
 .word 0x44444444, 0x55555555
 .word 0x55555555, 0x66666666

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11223344	0x55667788	...	...
...	...	...	...	...
0x20000100	0x11111111	0x22222222	0x33333333	0x44444444
0x20000110	0x55555555	0x66666666	...	...
...	...	...	...	...

Assembler-Direktiven

Beispiel:

...
tab1: .byte **0x11** 0x22, 0x33, 0x44
.byte 0x55, 0x66, 0x77, 0x88

...
tab2: .word 0x11111111, 0x22222222
.word 0x44444444, 0x55555555
.word 0x55555555, 0x66666666

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x 11 223344	0x55667788	...	...
...	...	...	...	...
0x20000100	0x11111111	0x22222222	0x33333333	0x44444444
0x20000110	0x55555555	0x66666666	...	...
...	...	...	...	...

Assembler-Direktiven

Beispiel:

...
tab1: .byte 0x11, 0x22, 0x33, 0x44
.byte 0x55, 0x66, 0x77, 0x88

...
tab2: .word 0x11111111, 0x22222222
.word 0x44444444, 0x55555555
.word 0x55555555, 0x66666666

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11223344	0x55667788	...	...
...	...	...	...	...
0x20000100	0x11111111	0x22222222	0x33333333	0x44444444
0x20000110	0x55555555	0x66666666	...	...
...	...	...	...	...

Assembler-Direktiven

Beispiel:

...
tab1: .byte 0x11, 0x22, 0x33, 0x44
 .byte 0x55, 0x66, 0x77, 0x88

...
tab2: .word 0x11111111, 0x22222222
 .word 0x44444444, 0x55555555
 .word 0x55555555, 0x66666666

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11223344	0x55667788	...	...
...	...	...	...	...
0x20000100	0x11111111	0x22222222	0x33333333	0x44444444
0x20000110	0x55555555	0x66666666	...	...
...	...	...	...	...

Assembler-Direktiven

Beispiel:

...
tab1: .byte 0x11, 0x22, 0x33, 0x44
 .byte 0x55, 0x66, 0x77, 0x88

...
tab2: .word 0x11111111, 0x22222222
 .word 0x44444444, 0x55555555
 .word 0x55555555, 0x66666666

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x11223344	0x55667788	...	...
...	...	...	...	...
0x20000100	0x11111111	0x22222222	0x33333333	0x44444444
0x20000110	0x55555555	0x66666666	...	...
...	...	...	...	...

Assembler-Direktiven

<label> .equ <data>

Textersetzung eines
Wertes durch einen
symbolischen Namen

Beispiel:

...

```
.equ COUNT, 0xFFFF  
.equ TAB, 0x20000000
```

...

```
loop:  mov    r0, #COUNT  
       ldr    r1, =#TAB
```

...

R0	0x0000FFFF
R1	0x20000000
R2	
R3	
R4	
R5	
R6	
R7	
R8	
...	

Assembler-Direktiven

<label> .equ <data>

Beispiel:

...

```
.equ COUNT, 0xFFFF  
.equ TAB, 0x20000000
```

...

```
loop:  mov  r0, #COUNT  
       ldr  r1, =#TAB
```

...

Textersetzung eines
Wertes durch einen
symbolischen Namen

R0	0x0000FFFF
R1	0x20000000
R2	
R3	
R4	
R5	
R6	
R7	
R8	
...	

Additionsbefehle

Beispiel: Tabellen zur Addition

@-----Operanden-Tabelle 1-----

tab1: **.word 0x00000001**, 0x00000002, 0x00000003
 .word 0x00000004, 0x00000005, 0x00000006

+

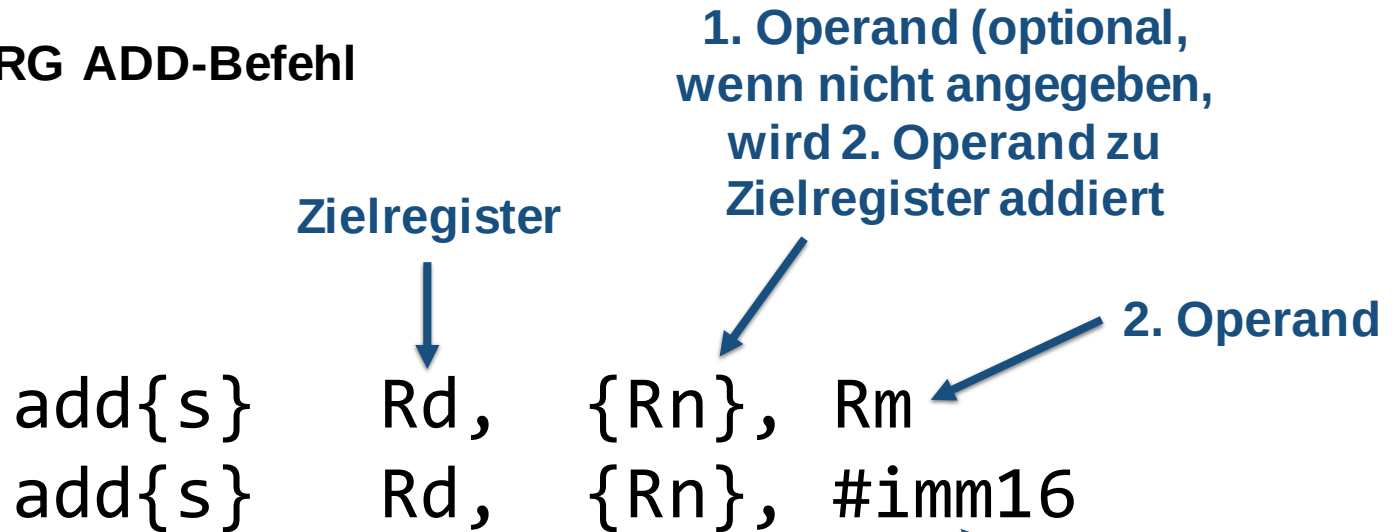
@-----Operanden-Tabelle 2-----

tab2: **.word 0x00000007**, 0x00000008, 0x00000009
 .word 0x0000000A, 0x0000000B, 0x0000000C

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x00000008	...	...	...
0x20000010	...	...	...	...
0x20000020	...	...	...	...
...	...	...	...	...

Befehlsgruppe *Addition, Subtraktion*

STM32L476RG ADD-Befehl



Beispiele:

```
add  r0,  r1,  r2
add  r0,  r1
add  r3,  #4
```

```
R0 = R1 + R2
R0 = R0 + R1
R0 = R0 + 4
```

maximal
16-bit Zahl

Befehlsgruppe *Addition, Subtraktion*

ADD, ADC, SUB, SBC, and RSB

Add, Add with Carry, Subtract, Subtract with Carry, and Reverse Subtract.

Syntax

```
op{S}{cond} {Rd,} Rn, Operand2
```

```
op{cond} {Rd,} Rn, #imm12; ADD and SUB only
```

Where:

- 'op' is one of the following:
 - ADD: Add
 - ADC: Add with carry
 - SUB: Subtract
 - SBC: Subtract with carry
 - RSB: Reverse subtract
- 'S' is an optional suffix. If S is specified, the condition code flags are updated on the result of the operation (see [Conditional execution on page 65](#))
- 'cond' is an optional condition code (see [Conditional execution on page 65](#))
- 'Rd' is the destination register. If Rd is omitted, the destination register is Rn
- 'Rn' is the register holding the first operand
- 'Operand2' is a flexible second operand (see [Flexible second operand on page 60](#) for details of the options)
- 'imm12' is any value in the range 0—4095

...

Befehlsgruppe *Addition, Subtraktion*

R0	...
R1	...
R2	0x20000000
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```
adr r0, op1
adr r1, op2
ldr r2, =0x20000000
loop: ldr r3, [r0]
      ldr r4, [r1]
      add r3, r4
      str r3, [r2]
```

```
op1: .word 0x00000003
op2: .word 0x00000004
```

Speicher

Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	...	...	...	...
...	...	...	...	...

Befehlsgruppe *Addition, Subtraktion*

R0	...
R1	...
R2	0x20000000
R3	0x00000003
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```
adr r0, op1
adr r1, op2
ldr r2, =0x20000000
loop: ldr r3, [r0]
      ldr r4, [r1]
      add r3, r4
      str r3, [r2]
```

```
op1: .word 0x00000003
op2: .word 0x00000004
```

Speicher

Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	...	...	...	...
...	...	...	...	...

Befehlsgruppe *Addition, Subtraktion*

R0	...
R1	...
R2	0x20000000
R3	0x00000003
R4	0x00000004
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```
adr r0, op1
adr r1, op2
ldr r2, =0x20000000
loop: ldr r3, [r0]
      ldr r4, [r1]
      add r3, r4
      str r3, [r2]
```

```
op1: .word 0x00000003
op2: .word 0x00000004
```

Speicher

Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	...	...	...	...
...	...	...	...	...

Befehlsgruppe *Addition, Subtraktion*

R0	...
R1	...
R2	0x20000000
R3	0x00000003
R4	0x00000004
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```
adr r0, op1
adr r1, op2
ldr r2, =0x20000000
loop: ldr r3, [r0]
      ldr r4, [r1]
      add r3, r4
      str r3, [r2]
```

```
op1: .word 0x00000003
op2: .word 0x00000004
```

```
0x00000003
+ 0x00000004
-----
0x00000007
```

Speicher

Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	...	...	...	...
...	...	...	...	...

Befehlsgruppe *Addition, Subtraktion*

R0	...
R1	...
R2	0x20000000
R3	0x00000007
R4	0x00000004
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```
adr r0, op1
adr r1, op2
ldr r2, =0x20000000
loop: ldr r3, [r0]
      ldr r4, [r1]
      add r3, r4
      str r3, [r2]
```

```
op1: .word 0x00000003
op2: .word 0x00000004
```

Speicher

Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	...	...	...	...
0x20000010	...	...	...	...
...	...	...	...	...

Befehlsgruppe *Addition, Subtraktion*

R0	...
R1	...
R2	0x20000000
R3	0x00000007
R4	0x00000004
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```
adr  r0, op1
adr  r1, op2
ldr  r2, =0x20000000
loop: ldr  r3, [r0]
      ldr  r4, [r1]
      add  r3, r4
      str  r3, [r2]
```

```
op1:  .word 0x00000003
op2:  .word 0x00000004
```

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x00000007	...	...	...
0x20000010	...	...	...	...
...	...	...	...	...

Zahlendarstellung

Zahlensysteme

Dezimal	Dual	Hexadezimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	10000	10
17	10001	11

Ziffern in Systemen verschiedener Basis

Dual → Hexadezimal:
vier Stellen zusammenfassen

Beispiel:
 $1100\ 0110_{(2)} \rightarrow 0xC6$

Zahlensysteme

Ziffern in Systemen verschiedener Basis

$$Z = a_n \cdot B^n + a_{n-1} \cdot B^{n-1} + \dots + a_1 \cdot B^1 + a_0 \cdot B^0$$

B: Basis des Zahlensystems mit $1 < B < \infty$

a_i : Ziffern mit $0 < a_i < B$

Dezimalsystem	Basis:	10
	Ziffernsymbole:	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Dualsystem	Basis:	2
	Ziffernsymbole:	0, 1
Hexadezimalsystem	Basis:	16
	Ziffernsymbole:	0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Rechnen mit Binärzahlen

Rechnen mit Binärzahlen

Beispiel: Addieren zweier 16-Bit-Zahlen in einem 8-Bit-System

		1.		2.	
Zahl 1:	0011	0111	1011	0100	
Zahl 2:	0010	0001	1011	1011	
	1	111	1	11	
	0101	1001	0110	1111	

Übertrag?

Befehlsgruppe *Addition, Subtraktion*

ADD, ADC, SUB, SBC, and RSB

Add, Add with Carry, Subtract, Subtract with Carry, and Reverse Subtract.

Syntax

`op{S}{cond} {Rd,} Rn, Operand2`

`op{cond} {Rd,} Rn, #imm12; ADD and SUB only`

Where:

- 'op' is one of the following:

ADD: Add

ADC: Add with carry

SUB: Subtract

SBC: Subtract with carry

RSB: Reverse subtract

- 'S' is an optional suffix. If S is specified, the condition code flags are updated on the result of the operation (see [Conditional execution on page 65](#))
- 'cond' is an optional condition code (see [Conditional execution on page 65](#))
- 'Rd' is the destination register. If Rd is omitted, the destination register is Rn
- 'Rn' is the register holding the first operand
- 'Operand2' is a flexible second operand (see [Flexible second operand on page 60](#) for details of the options)
- 'imm12' is any value in the range 0—4095

...

Addieren mit Carry

Positive und Negative Zahlen

Most Significant Bit (MSB)

0 = positive Zahl 1 = negative Zahl

Beispiele:

8 Bit: 01011101
15 **Vorzeichen**

16 Bit: 01100010 11111100
31 **Vorzeichen**

32 Bit: 10011111 01000001 10100011 00000110
Vorzeichen

Darstellung als Zweierkomplement-Zahl

Positive und Negative Zahlen

Darstellbare Bereiche ganzer Zahlen

Beispiel: 8-Bit-Zahlen

Dualzahlen	Hexadezimalzahlen	Dezimalzahlen
0111 1111	0x7F	127
0111 1110	0x7E	126
...	...	...
0000 0011	0x03	3
0000 0010	0x02	2
0000 0001	0x01	1
0000 0000	0x00	0
1111 1111	0xFF	-1
1111 1110	0xFE	-2
...	...	...
1000 0001	0x81	-127
1000 0000	0x80	-128

Positive und Negative Zahlen

Darstellbare Bereiche ganzer Zahlen

Beispiel: 8-Bit-Zahlen

	Dualzahlen	Hexadezimalzahlen	Dezimalzahlen
	0111 1111	0x7F	127
	0111 1110	0x7E	126
	...	...	...
	0000 0011	0x03	3
	0000 0010	0x02	2
	0000 0001	0x01	1
Vorzeichen →	0000 0000	0x00	0
	1111 1111	0xFF	-1
	1111 1110	0xFE	-2
	...	...	...
	1000 0001	0x81	-127
	1000 0000	0x80	-128

Zahlendarstellung

Zahlendarstellung

Zahlendarstellung

Zahlenkreis (8-Bit-Zahlen)

0x7F

0x7E

0x7D

...

0x03

0x02

0x01

0x00

0xFF

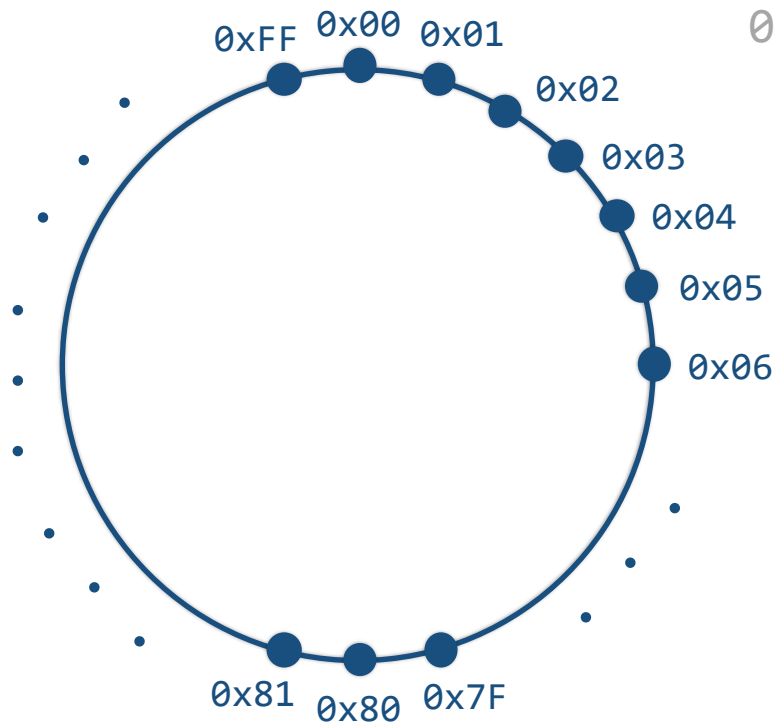
0xFE

0xFD

...

0x81

0x80



Beispiel:

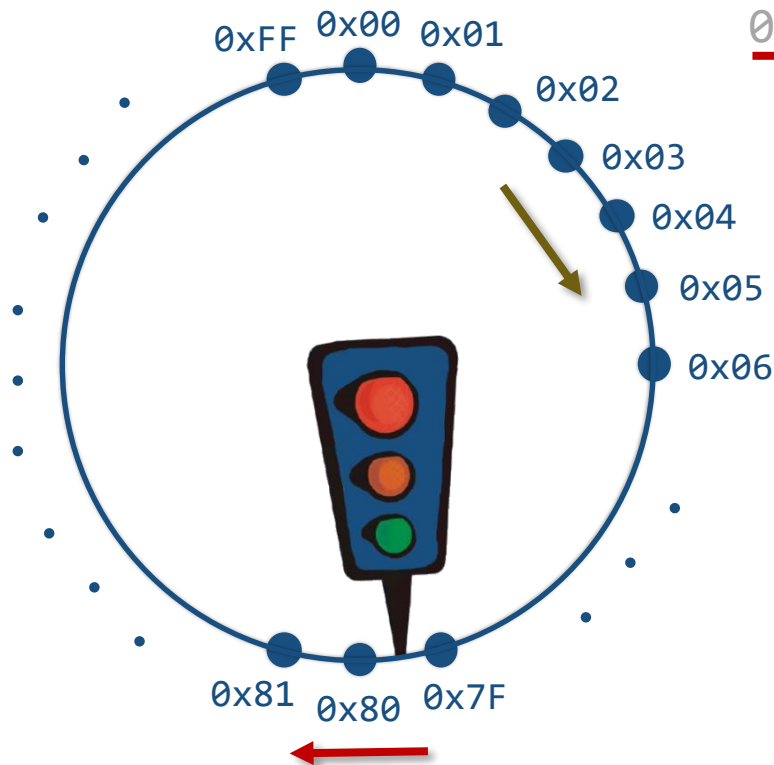
$0x04 + 0x01 \rightarrow 0x06$

$0x7F + 0x02 \rightarrow 0x81$

Zahlendarstellung

Zahlenkreis (8-Bit-Zahlen)

0x7F
0x7E
0x7D
...
0x03
0x02
0x01
0x00
0xFF
0xFE
0xFD
...
0x81
0x80



Beispiel:

$$\underline{0x04 + 0x01 \rightarrow 0x06}$$

$$\underline{0x7F + 0x02 \rightarrow 0x81}$$

Befehlsgruppe *Addition, Subtraktion*

ADD, ADC, SUB, SBC, and RSB

Add, Add with Carry, Subtract, Subtract with Carry, and Reverse Subtract.

Syntax

`op{S}{cond} {Rd,} Rn, Operand2`

`op{cond} {Rd,} Rn, #imm12; ADD and SUB only`

Where:

- 'op' is one of the following:
 - ADD: Add
 - ADC: Add with carry
 - SUB: Subtract
 - SBC: Subtract with carry
 - RSB: Reverse subtract
- 'S' is an optional suffix. If S is specified, the condition code flags are updated on the result of the operation (see [Conditional execution on page 65](#))
- 'cond' is an optional condition code (see [Conditional execution on page 65](#))
- 'Rd' is the destination register. If Rd is omitted, the destination register is Rn
- 'Rn' is the register holding the first operand
- 'Operand2' is a flexible second operand (see [Flexible second operand on page 60](#) for details of the options)
- 'imm12' is any value in the range 0—4095

...

Condition flags

If S is specified, these instructions update the N, Z, C and V flags according to the result.

Overflow-
Bit

Befehlsgruppe *Sprungbefehle*

Sprungbefehle, die die CCR Flags auswerten

beq	label	branch if equal
bne	label	branch if not equal
bgt	label	branch if greater than
bge	label	branch if greater or equal
blt	label	branch if less than
ble	label	branch if less or equal
bvs	label	branch if overflow set
bvc	label	branch if overflow clear
bcs	label	branch if carry set
bcc	label	branch if carry clear

Vergleichen und Auswertung der Flags

Befehlsgruppe *Sprungbefehle*

CMP and CMN

Compare and Compare Negative.

Syntax

`CMP{cond} Rn, Operand2`

`CMN{cond} Rn, Operand2`

Where:

- '*cond*' is an optional condition code (see [Conditional execution on page 65](#)).
- '*Rn*' is the register holding the first operand.
- '*Operand2*' is a flexible second operand (see [Flexible second operand on page 60](#)) for details of the options.

Operation

These instructions compare the value in a register with *operand2*. They update the condition flags on the result, but do not write the result to a register.

The CMP instruction subtracts the value of *operand2* from the value in *Rn*. This is the same as a SUBS instruction, except that the result is discarded.

The CMN instruction adds the value of *operand2* to the value in *Rn*. This is the same as an ADDS instruction, except that the result is discarded.

Restrictions

In these instructions:

- Do not use PC.
- *Operand2* must not be SP.

Condition flags

These instructions update the N, Z, C and V flags according to the result.

...

Befehlsgruppe *Sprungbefehle*

Condition flags

These instructions update the N, Z, C and V flags according to the result.

Objekte im Speicher oder in Registern werden nicht verändert

Flags (Condition Codes) werden verändert

Auswertung der Flags: Branch Conditionally

Befehlsgruppe *Sprungbefehle*

Sprungbefehle, die die CCR Flags auswerten

beq	label	branch if equal
bne	label	branch if not equal
bgt	label	branch if greater than
bge	label	branch if greater or equal
blt	label	branch if less than
ble	label	branch if less or equal
bvs	label	branch if overflow set
bvc	label	branch if overflow clear
bcs	label	branch if carry set
bcc	label	branch if carry clear

Auswertung der Flags

```
adr    r0, zahl
loop:  ldr    r1, [r0], #4
        cmp    r1, #0
        beq    ziel
        b      loop
```

...

```
ziel:  ...
```

...

```
zahl:  .word 0x00000003, 0x00000000
```

R0	...
R1	0x00000003
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Auswertung der Flags

```
adr    r0, zahl
loop:  ldr    r1, [r0], #4
       cmp    r1, #0 ← Inhalt von R1 == 0?
       beq    ziel
       b      loop

...

ziel:  ...

...

zahl:  .word 0x00000003, 0x00000000
```

R0	...
R1	0x00000003
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Auswertung der Flags

```
    adr    r0, zahl
loop: ldr    r1, [r0], #4
      cmp   r1, #0
      beq   ziel ← wenn ja,
                  zu ziel
      b     loop

...

ziel:  ...

...

zahl:  .word 0x00000003, 0x00000000
```

R0	...
R1	0x00000003
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Auswertung der Flags

```
adr    r0, zahl
loop:  ldr    r1, [r0], #4
        cmp    r1, #0
        beq    ziel
        b      loop
```

...

```
ziel:  ...
```

...

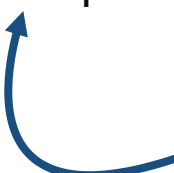
```
zahl:  .word 0x00000003, 0x00000000
```

$0x00000003 \neq 0$

R0	...
R1	0x00000003
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Auswertung der Flags

```
        adr    r0, zahl
loop:    ldr    r1, [r0], #4
        cmp    r1, #0
        beq    ziel
        b      loop
```



...

ziel: ...

...

```
zahl:   .word  0x00000003, 0x00000000
```

R0	...
R1	0x00000003
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Auswertung der Flags

```
adr    r0, zahl
loop:  ldr    r1, [r0], #4
        cmp    r1, #0
        beq    ziel
        b      loop
```

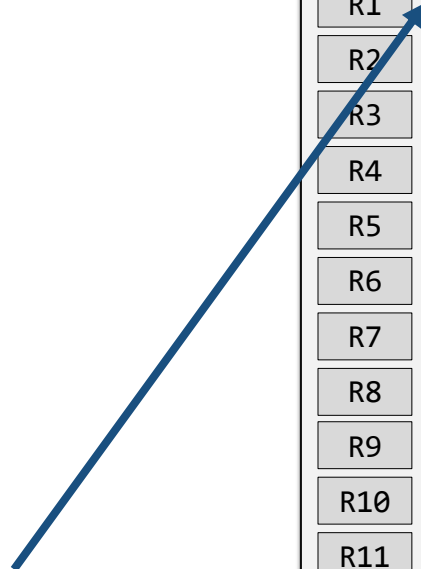
...

```
ziel:  ...
```

...

```
zahl:  .word 0x00000003, 0x00000000
```

R0	...
R1	0x00000000
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	



Auswertung der Flags

```
adr    r0, zahl
loop:  ldr    r1, [r0], #4
        cmp   r1, #0 ← Inhalt von R1 == 0?
        beq   ziel
        b     loop
```

...

ziel: ...

...

```
zahl:  .word 0x00000003, 0x00000000
```

R0	...
R1	0x00000000
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Auswertung der Flags

```
    adr    r0, zahl
loop: ldr    r1, [r0], #4
      cmp   r1, #0
      beq   ziel ← wenn ja,
                  zu ziel
      b     loop

...

ziel:  ...

...

zahl:  .word 0x00000003, 0x00000000
```

R0	...
R1	0x00000000
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Auswertung der Flags

```
adr  r0, zahl
loop: ldr  r1, [r0], #4
      cmp  r1, #0
      beq  ziel
      b    loop
```

...

```
ziel: ...
```

...

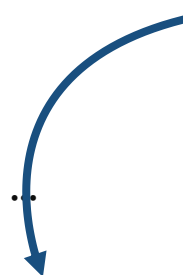
```
zahl: .word 0x00000003, 0x00000000
```

$0x00000000 = 0$

R0	...
R1	0x00000000
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Auswertung der Flags

```
adr    r0, zahl
loop:  ldr    r1, [r0], #4
        cmp   r1, #0
        beq   ziel
        b     loop
...
ziel:  ...
...
zahl:  .word  0x00000003, 0x00000000
```



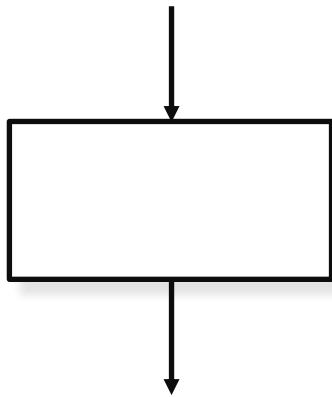
R0	...
R1	0x00000000
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Flussdiagramme

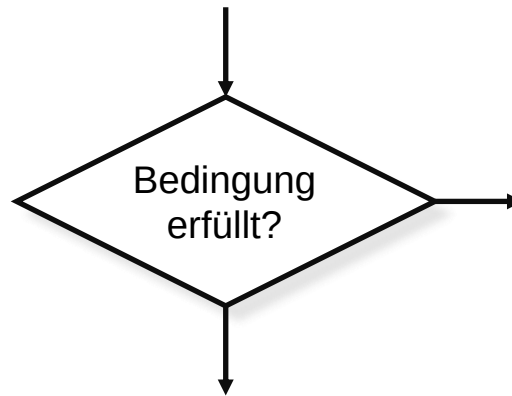
Auswertung der Flags

Grundelemente

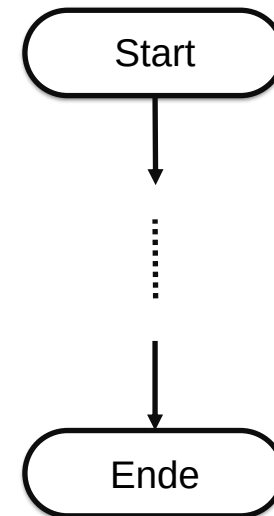
Anweisung



Verzweigung



Übergangsstelle
(Anfang, Ende)



Auswertung der Flags

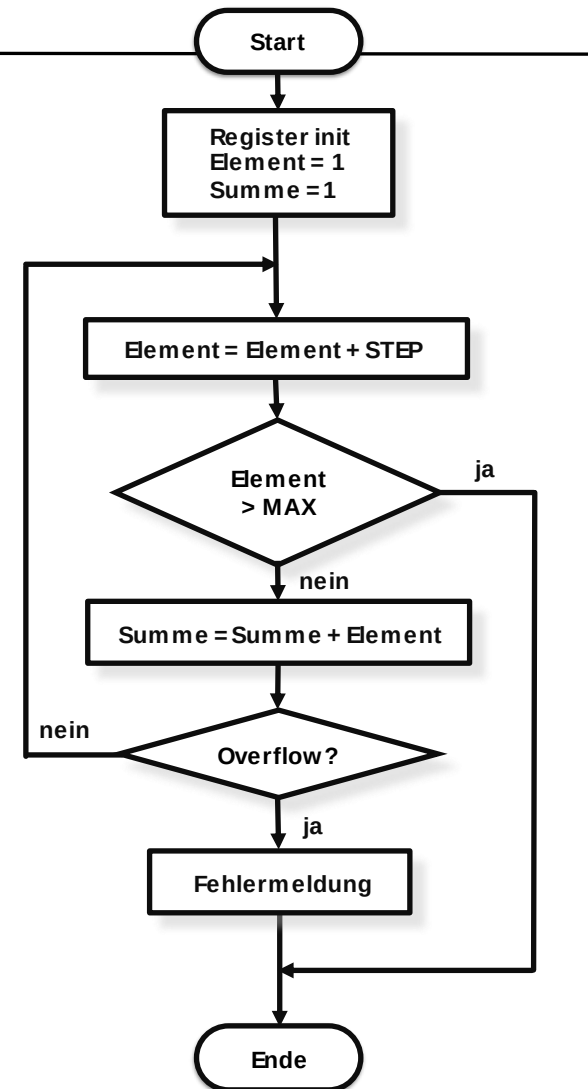
Aufgabe

Schreiben Sie ein Programm, das die Zahlen einer Zahlenreihe (32 Bit Zahlen) von 1 bis zur Obergrenze MAX addieren kann. Das Ergebnis steht in der Variablen summe.

Zur Erzeugung der Einzelelemente dieser Zahlenreihe wird die Konstante STEP benutzt (neues Reihenelement = altes Reihenelement + STEP).

Beispiel:

für MAX= 8 und STEP= 2 werden die Zahlen 1, 3, 5 und 7 addiert. Das Ergebnis für dieses Beispiel ist die Zahl 16.

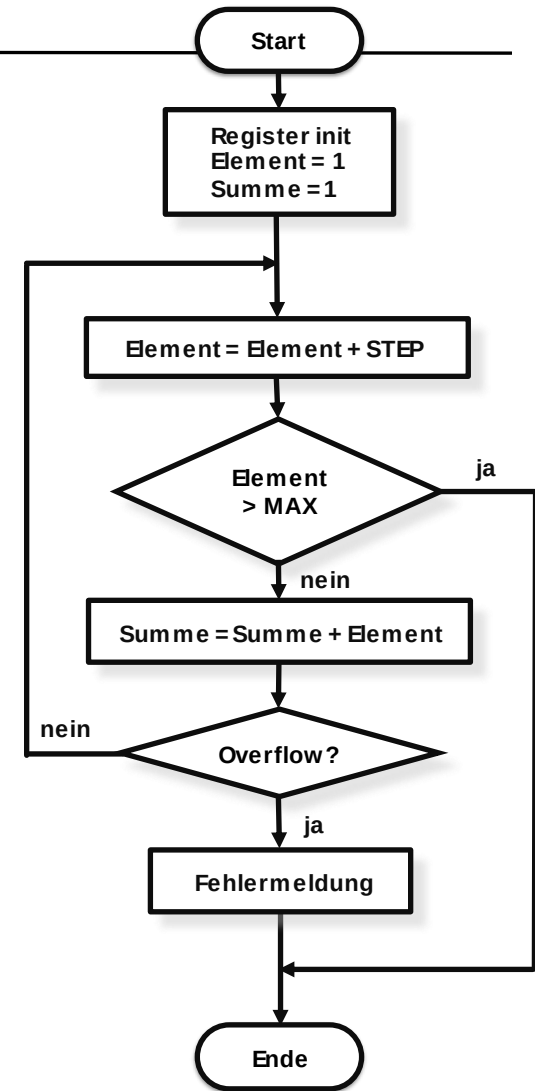


Auswertung der Flags

```
.equ MAX, 8  
.equ STEP, 2
```

```
ldr r0, =0x20000000  
mov r1, #1 @ Startelement  
mov r2, #1 @ Startsumme  
loop: add r1, #STEP @ neues Reihenelement  
cmp r1, #MAX @ Element > MAX?  
bgt ende @ wenn ja, zu ende  
adds r2, r1 @ neue Summe  
bvs err @ Overflow?  
str r2, [r0] @ Summe speichern  
b loop @ Schleife  
err: mov r4, #0xFFFF @ Fehlerkennung  
str r4, [r0] @ speichern
```

```
@ Ende des Programms  
@ Stop durch Endlos-Schleife  
ende: nop  
b ende
```



Schleifenkontrolle

Schleifenkontrolle

Beispiel: Schleifenkontrolle mit Register

```
.equ    COUNT, 0x12
```

```
...
```

```
loop:   ldr    r0,    =#COUNT
        ...
        subs   r0,    #1
        bne    loop
```



```
...
```

Anzahl Schleifendurchläufe?

Schleifenkontrolle

Beispiel: Schleifenkontrolle mit Register

```
.equ    COUNT, 0x12
```

```
...
```

```
    ldr    r0,    =#COUNT
```

```
loop:
```

```
    ...
```

```
    subs   r0,    #1
```

```
    bne    loop
```

```
...
```

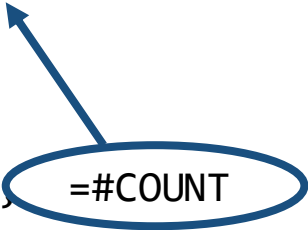
Schleifenzähler
initialisieren



Schleifenkontrolle

Beispiel: Schleifenkontrolle mit Register

```
.equ    COUNT, 0x12
...
loop:   ldr    r0,    =#COUNT
        ...
        subs   r0,    #1
        bne    loop
...
```



Schleifenkontrolle

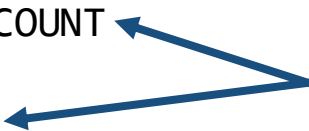
Beispiel: Schleifenkontrolle mit Register

```
.equ    COUNT, 0x12
```

```
...
```

```
loop:   ldr    r0,    =#COUNT  
        ...  
        subs   r0,    #1  
        bne    loop
```

Schleifenkontrolle



```
...
```

Schleifenkontrolle

Beispiel: Schleifenkontrolle mit Register

```
.equ    COUNT, 0x12
```

```
...
```

```
loop:   ldr    r0,    =#COUNT
        ...
        subs   r0,    #1
        bne    loop
```

```
...
```

Schleife wiederholen, bis
Schleifenzähler gleich Null



Addition von Zahlen größer 32 Bit

ADD / SUB Befehle

ADC-Befehl

ADD, ADC, SUB, SBC, and RSB

Add, Add with Carry, Subtract, Subtract with Carry, and Reverse Subtract.

Syntax

```
op{S}{cond} {Rd,} Rn, Operand2  
op{cond} {Rd,} Rn, #imm12; ADD and SUB only
```

Where:

- 'op' is one of the following:

ADD: Add

ADC: Add with carry

SUB: Subtract

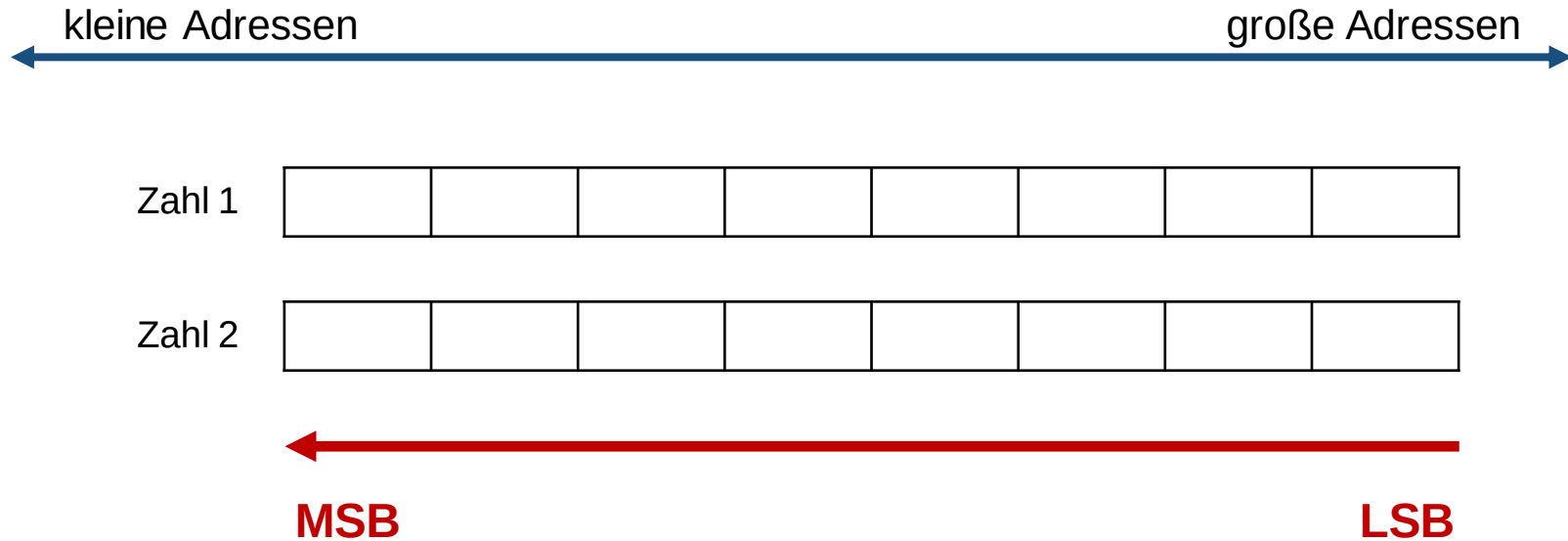
SBC: Subtract with carry

RSB: Reverse subtract

- 'S' is an optional suffix. If S is specified, the condition code flags are updated on the result of the operation (see [Conditional execution on page 65](#))
- 'cond' is an optional condition code (see [Conditional execution on page 65](#))
- 'Rd' is the destination register. If Rd is omitted, the destination register is Rn
- 'Rn' is the register holding the first operand
- 'Operand2' is a flexible second operand (see [Flexible second operand on page 60](#) for details of the options)
- 'imm12' is any value in the range 0—4095

...

ADD / SUB Befehle



ADD / SUB Befehle

Adressierungsart Pre-Increment

R0	0x20000000
R1	0x20000050
R2	
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

Vorbereitung:
Register mit Adressen initialisieren

Referenz auf...

Speicher				
Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x12345678	0x9ABCDEF0	...	...
0x20000010	...	...	...	...
...	...	...	...	...
0x20000050	0xFFFFFFFF	0xFFFFFFFF	...	...
...	...	...	...	...

ADD / SUB Befehle

Adressierungsart Pre-Increment

R0	0x20000000
R1	0x20000050
R2	
R3	
R4	
R5	
R6	
R7	0x9ABCDEF0
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```
ldr r7, [r0, #4]
```

```
ldr r8, [r1, #4]
```

Referenz auf (R0 + 4)

Speicher

Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x12345678	0x9ABCDEF0	...	...
0x20000010	...	...	...	...
...	...	...	...	...
0x20000050	0xFFFFFFFF	0xFFFFFFFF	...	...
...	...	...	...	...

ADD / SUB Befehle

Adressierungsart Pre-Increment

R0	0x20000000
R1	0x20000050
R2	
R3	
R4	
R5	
R6	
R7	0x9ABCDEF0
R8	0xFFFFFFFF
R9	
R10	
R11	
R12	
SP	
LR	
PC	

```
ldr r7, [r0, #4]
```

```
ldr r8, [r1, #4]
```

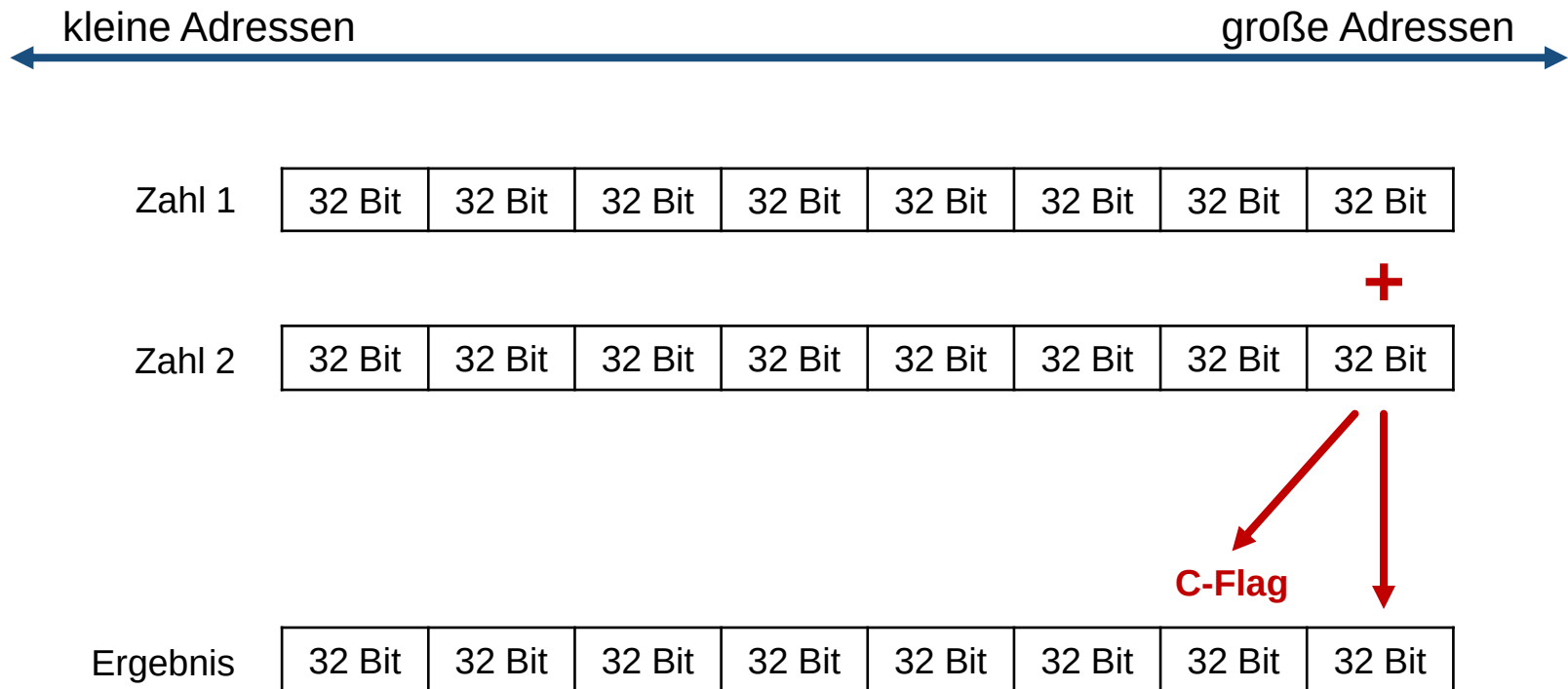
Referenz auf (R1 + 4)

Speicher

Address	0 - 3	4 - 7	8 - B	C - F
0x20000000	0x12345678	0x9ABCDEF0	...	...
0x20000010	...	...	...	...
...	...	...	...	...
0x20000050	0xFFFFFFFF	0xFFFFFFFF	...	...
...	...	...	...	...

ADD / SUB Befehle

Addieren von Zahlen mit einer Datenbreite größer 32 Bit

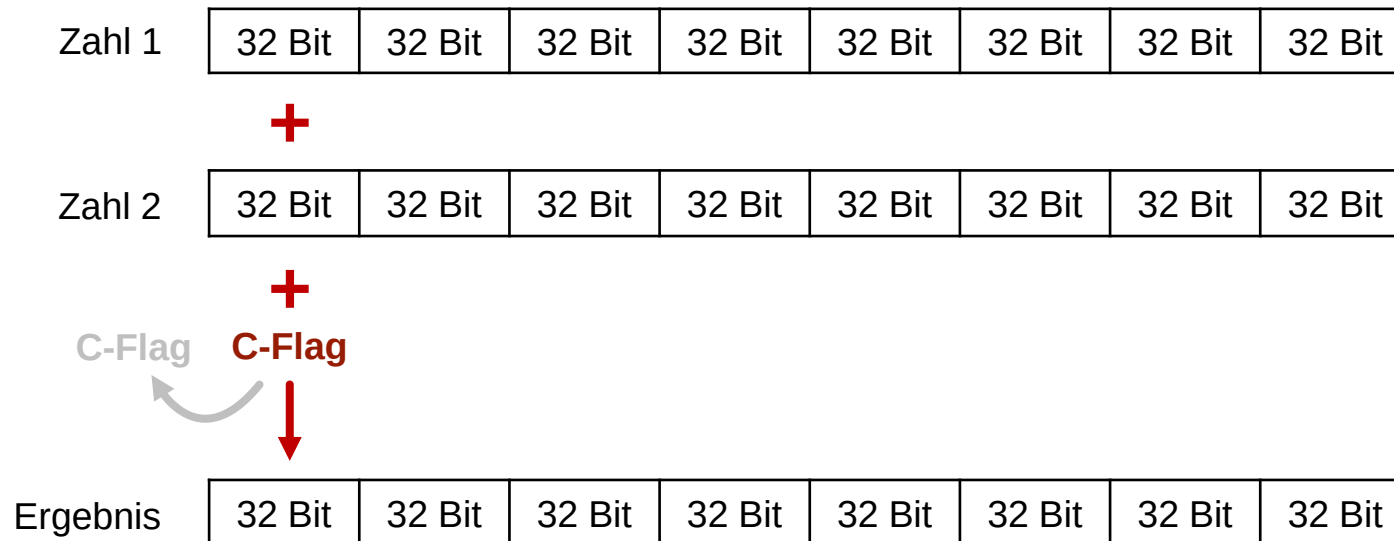


ADD / SUB Befehle

Addieren von Zahlen mit einer Datenbreite größer 32 Bit

kleine Adressen

große Adressen

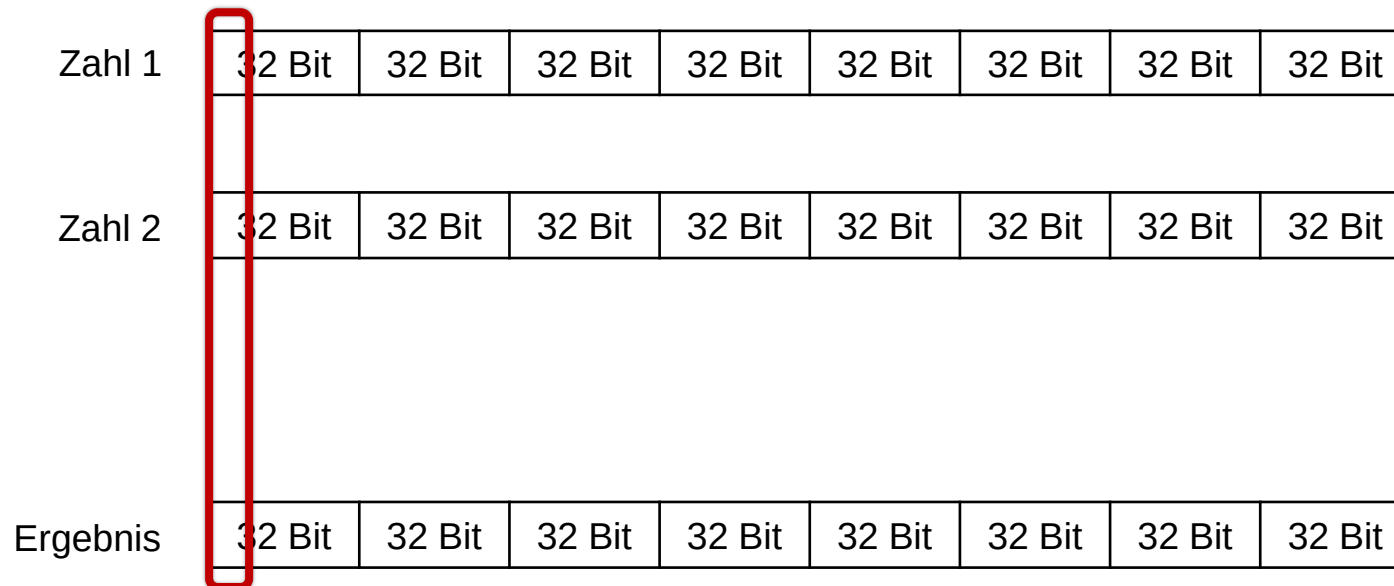


ADD / SUB Befehle

Addieren von Zahlen mit einer Datenbreite größer 32 Bit

kleine Adressen

große Adressen



Vorzeichen

ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr    r0, =0x20000000
adr    r1, zahl1
adr    r2, count
adr    r3, count
ldr    r4, [r1, #-4]!
ldr    r5, [r2, #-4]!
adds   r4, r5
str    r4, [r0, #12]!
loop:  ldr    r4, [r1, #-4]!
       ldr    r5, [r2, #-4]!
       adcs   r4, r5
       str    r4, [r0, #-4]!
       sub    r3, #1
       cmp    r3, #0
       bne    loop
```

Register initialisieren

Register R1

Register R2, R3

```
zahl1:  .word  0x0001A832, 0x8C530904, 0x12345678, 0xABCDEF12
zahl2:  .word  0x037DEF07, 0x3599F1C1, 0x34503048, 0x85492742
count:  .word  0x00000003
```


ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr r0, =0x20000000
adr r1, zahl1
adr r2, count
adr r3, count
ldr r4, [r1, #-4]!
ldr r5, [r2, #-4]!
adds r4, r5
str r4, [r0, #12]!
loop: ldr r4, [r1, #-4]!
      ldr r5, [r2, #-4]!
      adcs r4, r5
      str r4, [r0, #-4]!
      sub r3, #1
      cmp r3, #0
      bne loop
```

} Werte für die erste Addition in Register R4 und R5 laden, R1 und R2 werden verändert

```
zahl1: .word 0x0001A832, 0x8C530904, 0x12345678, 0xABCDEF12
zahl2: .word 0x037DEF07, 0x3599F1C1, 0x34503048, 0x85492742
count: .word 0x00000003
```

Register R1

Register R2

ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr    r0, =0x20000000
adr    r1, zahl1
adr    r2, count
adr    r3, count
ldr    r4, [r1, #-4]!
ldr    r5, [r2, #-4]!
adds   r4, r5
loop:  ldr    r4, [r1, #-4]!
       ldr    r5, [r2, #-4]!
       adcs   r4, r5
       str    r4, [r0, #12]!
       sub    r3, r3, r4
       cmp    r3, r4
       bne    loop
```

} erste von vier Additionen ohne Carry, Carry wird erzeugt

ADD, ADC, SUB, SBC, and RSB

Add, Add with Carry, Subtract, Subtract with Carry, and Reverse Subtract.

Syntax

op{S}{cond} {Rd}, {Rn}, Operand2

op{cond} {Rd}, {Rn}, #imm12; ADD and SUB only

```
zahl1: .word 0x0001A832, 0x8C530904, 0x12345678, 0xABCDEF12
zahl2: .word 0x037DEF07, 0x3599F1C1, 0x34503048, 0x85492742
count: .word 0x00000003
```

ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr    r0, =0x20000000
adr    r1, zahl1
adr    r2, count
adr    r3, count
ldr    r4, [r1, #-4]!
ldr    r5, [r2, #-4]!
adds   r4, r5
loop:  str    r4, [r0, #12]!  } Wert speichern an R0 + 12 = 0x2000000C
      ldr    r4, [r1, #-4]!
      ldr    r5, [r2, #-4]!
      adcs   r4, r5
      str    r4, [r0, #-4]!
      sub    r3, #1
      cmp    r3, #0
      bne    loop
```

```
zahl1:  .word  0x0001A832, 0x8C530904, 0x12345678, 0xABCDEF12
zahl2:  .word  0x037DEF07, 0x3599F1C1, 0x34503048, 0x85492742
count:  .word  0x00000003
```

ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr    r0, =0x20000000
adr    r1, zahl1
adr    r2, count
adr    r3, count
ldr    r4, [r1, #-4]!
ldr    r5, [r2, #-4]!
adds   r4, r5
loop:  str    r4, [r0, #12]!
      ldr    r4, [r1, #-4]!
      ldr    r5, [r2, #-4]!
      adcs   r4, r5
      str    r4, [r0, #-4]!
      sub    r3, #1
      cmp    r3, #0
      bne    loop
```

Werte für die nächste Addition in Register R4 und R5 laden, R1 und R2 werden verändert

```
zahl1:  .word  0x0001A832, 0x8C530904, 0x12345678, 0xABCDEF12
zahl2:  .word  0x037DEF07, 0x3599F1C1, 0x34503048, 0x85492742
count:  .word  0x00000003
```

Register R1

Register R2

ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr    r0, =0x20000000
adr    r1, zahl1
adr    r2, count
adr    r3, count
ldr    r4, [r1, #0]
ldr    r5, [r2, #0]
adds   r4, r5
str    r4, [r1, #4]
loop:  ldr    r4, [r1, #-4]!
        ldr    r5, [r2, #-4]!
        adcs   r4, r5
        str    r4, [r0, #-4]!
        sub    r3, #1
        cmp    r3, #0
        bne    loop
```

ADD, ADC, SUB, SBC, and RSB

Add, Add with Carry, Subtract, Subtract with Carry, and Reverse Subtract.

Syntax

op{S} cond {Rd}, Rn, Operand2

op{cond} {Rd}, Rn, #imm12; ADD and SUB only

} zweite bis vierte von vier Additionen mit Carry,
neues Carry wird erzeugt

```
zahl1:  .word  0x0001A832, 0x8C530904, 0x12345678, 0xABCDEF12
zahl2:  .word  0x037DEF07, 0x3599F1C1, 0x34503048, 0x85492742
count:  .word  0x00000003
```

ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr    r0, =0x20000000
adr    r1, zahl1
adr    r2, count
adr    r3, count
ldr    r4, [r1, #-4]!
ldr    r5, [r2, #-4]!
adds   r4, r5
str    r4, [r0, #12]!
loop:  ldr    r4, [r1, #-4]!
       ldr    r5, [r2, #-4]!
       adcs   r4, r5
       str    r4, [r0, #-4]! } Wert speichern an R0 – 4
       sub    r3, #1
       cmp    r3, #0
       bne    loop
```

```
zahl1: .word  0x0001A832, 0x8C530904, 0x12345678, 0xABCDEF12
zahl2: .word  0x037DEF07, 0x3599F1C1, 0x34503048, 0x85492742
count: .word  0x00000003
```

ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr    r0, =0x20000000
adr    r1, zahl1
adr    r2, count
adr    r3, count
ldr    r4, [r1, #-4]!
ldr    r5, [r2, #-4]!
adds   r4, r5
str    r4, [r0, #12]!
loop:  ldr    r4, [r1, #-4]!
       ldr    r5, [r2, #-4]!
       adcs   r4, r5
       str    r4, [r0, #-4]!
       sub    r3, #1
       cmp    r3, #0
       bne    loop
```

} count in R3 dekrementieren

```
zahl1: .word  0x0001A832, 0x8C530904, 0x12345678, 0xABCDEF12
zahl2: .word  0x037DEF07, 0x3599F1C1, 0x34503048, 0x85492742
count: .word  0x00000003
```

ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr    r0, =0x20000000
adr    r1, zahl1
adr    r2, count
adr    r3, count
ldr    r4, [r1, #-4]!
ldr    r5, [r2, #-4]!
adds   r4, r5
str    r4, [r0, #12]!
loop:  ldr    r4, [r1, #-4]!
       ldr    r5, [r2, #-4]!
       adcs   r4, r5
       str    r4, [r0, #-4]!
       sub    r3, #1
       cmp    r3, #0
       bne    loop
```

} wenn R3 ≠ 0, zurück zu loop

```
zahl1: .word  0x0001A832, 0x8C530904, 0x12345678, 0xABCDEF12
zahl2: .word  0x037DEF07, 0x3599F1C1, 0x34503048, 0x85492742
count: .word  0x00000003
```


ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr r0, =0x20000000
adr r1, zahl1
adr r2, count
adr r3, count
ldr r4, [r1, #-4]!
ldr r5, [r2, #-4]!
adds r4, r5
str r4, [r0, #12]!
loop: ldr r4, [r1, #-4]!
      ldr r5, [r2, #-4]!
      adcs r4, r5
      str r4, [r0, #-4]!
      sub r3, #1
      cmp r3, #0
      bne loop
```

**Carry-Flag darf in der Schleife
nicht verändert werden!**

```
zahl1: .word 0x0001A832, 0x8C530904, 0x12345678, 0xABCDEF12
zahl2: .word 0x037DEF07, 0x3599F1C1, 0x34503048, 0x85492742
count: .word 0x00000003
```

ADD / SUB Befehle

Beispiel: Addition zweier 128-bit Zahlen

```
ldr r0, =0x20000000
adr r1,
adr r2,
adr r3,
ldr r4,
ldr r5,
adds r4,
str r4,
loop: ldr r4,
      ldr r5,
      adcs r4,
      str r4,
      sub r3,
      cmp r3,
      bne loc
zahl1: .word
zahl2: .word
count: .word
```

Mnemonic	Operands	Brief description	Flags	Page
:				
CBNZ	Rn, label	Compare and branch if non zero	—	3.9.6 on page 144
:				
LDR	Rt, [Rn, #offset]	Load register with word	—	3.4 on page 69
:				
STR	Rt, [Rn, #offset]	Store register word	—	3.4 on page 69
:				
SUB, SUBW	{Rd,} Rn, #imm12	Subtract	N,Z,C,V	3.5.1 on page 83

ADD, ADDS, SUB, SBC, and RSB

Add, Add with Carry, Subtract, Subtract with Carry, and Reverse Subtract.

Syntax

op{S}[cond]: {Rd,} Rn, Operand2

op{cond}: {Rd,} Rn, #imm12; ADD and SUB only

'S' is an optional suffix. If S is specified, the condition code flags are updated on the result of the operation (see [Conditional execution on page 65](#))

ADD / SUB Befehle

Einerkomplement und Zweierkomplement

- Einerkomplement invertiert alle Bits
- Zweierkomplement invertiert alle Bits und addiert 1

ADD / SUB Befehle

Einerkomplement und Zweierkomplement

- Einerkomplement invertiert alle Bits
- Zweierkomplement invertiert alle Bits und addiert 1

MOV and MVN

Move and Move NOT.

Syntax

```
MOV{S}{cond} Rd, Operand2
```

```
MOV{cond} Rd, #imm16
```

```
MVN{S}{cond} Rd, Operand2
```

⋮

The MVN instruction takes the value of *operand2*, performs a bitwise logical NOT operation on the value, and places the result into *Rd*.

ADD / SUB Befehle

Einerkomplement

`movn r2, r0`

R0	0x0000000F
R1	
R2	0xFFFFFFFF
R3	
R4	
R5	
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	



ADD / SUB Befehle

Einerkomplement

```
movn r1, r0
```

Zweierkomplement

```
movn r5, r0
```

```
add r5, #1
```

R0	0x0000000F
R1	
R2	0xFFFFFFFF0
R3	
R4	
R5	0xFFFFFFFF1
R6	
R7	
R8	
R9	
R10	
R11	
R12	
SP	
LR	
PC	

